Enhancing Text-to-Cypher Capabilities of Small Language Models via Draft-Guided Schema Linking and Self-Correction

Tien Dat Nguyen[1,2], Nguyen Duy Linh Vu[1,2], Gia Kiet Le[1,2], Minh Nhut Nguyen[1,2], Dinh Thuan Nguyen[1,2*]

[1]Faculty of Information Systems, University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam.
[2]Vietnam National University, Ho Chi Minh City, Vietnam.

*Corresponding author(s). E-mail(s): thuannd@uit.edu.vn;
Contributing authors: 22520226@gm.uit.edu.vn; 22520780@gm.uit.edu.vn; 21522255@gm.uit.edu.vn; nhutnm@uit.edu.vn;

Abstract

Text-to-Cypher translation enables intuitive access to graph databases using natural language, yet open-source Small Language Models (SLMs) continue to face difficulties with complex schemas and limited context windows. This paper introduces a training-free two-phase workflow to enhance the Text-to-Cypher performance of SLMs. The first phase employs draft-based schema linking, where an initial query draft is generated from the full schema and analyzed via regex-based word boundary matching to extract mentioned node labels, constructing a focused sub-schema that reduces context length and eliminates irrelevant elements. The second phase integrates execution-driven self-correction, refining queries through an iterative feedback loop that leverages direct Neo4j runtime errors, augmented by automatic chain-of-thought hints providing targeted guidance for common Cypher pitfalls—including aliasing, clause ordering, and pattern validity—without requiring additional LLM calls. Evaluated on the Text2Cypher-2024v1 dataset across Qwen2.5-7B-Instruct, Qwen2.5-Coder-14B-Instruct, and fine-tuned variants (Gemma-2-9B and Qwen2.5-7B), our pipeline delivers absolute gains of up to 8.3 percentage points in Execution Accuracy over zero-shot full-schema baselines while cutting syntax errors by approximately 70% (from 25.6% to 7.3%) for base models, all on consumer-grade hardware. While schema linking benefits base models significantly, fine-tuned models show reduced

gains due to schema memorization effects; however, self-correction remains effective across all model types. Ablation studies reveal complementary benefits: schema linking mitigates hallucinations in base models, while self-correction tackles syntactic and structural issues universally. Though assessed on a subset of executable Neo4j instances due to benchmark constraints, the training-free design offers a practical, resource-efficient solution for deploying open-source SLMs in real-world graph querying scenarios.

Keywords: Database schema linking, sub-schema extraction, iterative self-correction, natural language processing, open-source models, Small Language Models (SLMs), Text-to-Cypher

1 Introduction

The widespread adoption of graph databases, particularly Neo4j, in domains such as biomedical research, social networks, and recommendation systems has driven demand for intuitive query interfaces. While Cypher offers powerful graph traversal capabilities, its complexity limits accessibility for non-technical users. Consequently, Text-to-Cypher—the translation of natural language questions into executable Cypher queries—has emerged as a critical research area, with growing interest driven by recent advancements in Graph Retrieval-Augmented Generation (Graph RAG) systems [1]. In Graph RAG architectures, knowledge graphs provide structured external knowledge to ground Large Language Models (LLMs), addressing limitations of traditional vector-based retrieval that often overlooks relational structures. Accurate Text-to-Cypher translation is thus increasingly vital for effective graph-based augmentation. While the Text-to-SQL domain benefits from mature open-source solutions and comprehensive benchmarks, Text-to-Cypher has only recently seen rapid progress through dedicated datasets [2], synthetic data generation [3, 4], and schema filtering techniques [5]. Nevertheless, existing models still encounter significant challenges with large or ambiguous schemas, frequently producing hallucinated node labels or relationship types, syntactically invalid queries, and errors in complex traversal patterns. Recent work in Text-to-SQL has demonstrated the effectiveness of schema linking [6, 7] and execution-driven self-correction [7, 8] as separate enhancement techniques. However, their systematic integration and adaptation to Text-to-Cypher—particularly for addressing graph-specific constraints such as relationship directionality, variable-length paths, and pattern matching—remains unexplored.

1.1 Core Challenges

Despite rapid progress in Text-to-Cypher translation driven by recent datasets and advanced fine-tuning techniques [2, 4], achieving robust performance in real-world scenarios remains difficult, particularly when deploying models in resource-constrained environments or using smaller open-source language models that prioritize efficiency and privacy. The key challenges that hinder reliable graph query generation include:

- **Schema Complexity and Alignment:** SLMs struggle to interpret large or ambiguous graph schemas due to limited parametric knowledge, frequently misaligning node labels, relationship types, and properties with natural language references.
- **Context Window Limitations:** Shorter context windows (typically 4k–8k tokens) in most SLMs restrict the ability to incorporate complete schema descriptions, few-shot examples, and complex queries simultaneously, resulting in incomplete contextual understanding and higher error rates.
- **Hallucinations and Syntactic Errors:** SLMs are prone to generating non-existent schema elements or syntactically invalid Cypher statements, especially when handling advanced constructs such as variable-length paths, conditional aggregations, or nested expressions.
- **Limited Error Correction Capabilities:** Without built-in verification mechanisms, these models struggle to incorporate feedback from graph engine execution errors, often requiring multiple external retries or human intervention to produce executable queries.

1.2 Proposed Approach

To address the aforementioned challenges, we propose a training-free, two-phase workflow that enhances Text-to-Cypher performance of open-source SLMs by adapting and integrating proven techniques from Text-to-SQL research to the unique constraints of graph databases. This approach remains feasible on consumer-grade hardware while delivering substantial improvements in query accuracy and executability.

1.2.1 Schema Linking via Draft Query Extraction

To handle large or ambiguous graph schemas without truncation or excessive context, we adapt the draft-based schema linking strategy from Text-to-SQL systems [6] to Cypher generation. We first prompt the model to generate a draft query using the full schema and the natural language question, then apply lightweight regular expressions to extract mentioned node labels and relationship types, including graph-specific patterns such as relationship directionality and variable-length paths. These extracted elements define a focused sub-schema, which is then used to re-prompt the model for generating the refined query. This narrows the context to relevant schema components while preserving structural constraints inherent to graph traversal, thereby reducing misalignment errors and hallucinations.

1.2.2 Cypher Generation and Iterative Self-Correction

Once the refined schema context is available, the model generates an initial query based on the user question. To ensure executability, we incorporate an iterative self-correction mechanism adapted from execution-driven approaches in Text-to-SQL [7, 8]. The generated query is validated for basic syntactic correctness using rule-based checks tailored to Cypher constraints, then executed directly against the Neo4j graph database. If execution fails, the error message is analyzed to provide automatic chain-of-thought hints that offer targeted diagnostic guidance for common Cypher error categories—including

syntax violations, unknown labels or properties, aliasing requirements, and pattern structure issues—without requiring additional LLM calls. The model is then prompted to revise the query using both the error message and the automatic hint. This feedback loop repeats until the query executes successfully or a predefined maximum number of iterations is reached, substantially improving execution accuracy.

1.3 Key Contributions

While schema linking and execution-driven self-correction have proven effective in Text-to-SQL translation, their systematic integration and adaptation to Text-to-Cypher—particularly for addressing graph-specific constraints such as relationship directionality, variable-length paths, and pattern matching—remains limited. This work addresses this gap through the following contributions:

- We present a unified, training-free enhancement pipeline that combines draft-based schema linking with execution-driven self-correction for Cypher query generation. The schema linking component leverages the model's own draft query as a relevance signal: by generating an initial Cypher query from the full schema and extracting mentioned node labels via keyword matching, we construct a focused sub-schema that reduces context length and eliminates irrelevant elements. For compact schemas (≤ 3 node labels), we bypass filtering to preserve complete context.
- We introduce automatic chain-of-thought (CoT) hints, a novel error-guidance mechanism specifically designed for Cypher query correction. Unlike generic self-correction approaches that simply feed raw error messages back to the model, our method analyzes execution errors using lightweight string matching and automatically generates structured diagnostic reasoning tailored to four common Cypher error categories: syntax violations, unknown labels or properties, aliasing requirements, and pattern structure issues. This targeted guidance enables effective error correction without additional LLM calls, representing a key innovation over existing self-correction techniques.
- We conduct comprehensive empirical evaluation across multiple open-source SLMs, demonstrating significant gains for base models in Execution Accuracy (up to 8.3 percentage points), BLEU, ROUGE-L, and Token F1. Our analysis uncovers a crucial trade-off for fine-tuned specialists: full-schema memorization competes with dynamic schema filtering, leading to performance degradation when schema linking is applied. Ablation studies confirm that schema linking reduces schema-level hallucinations while self-correction addresses syntactic errors, with syntax error rates dropping by approximately 70% (from 25.6% to 7.3%) for base models.
- We provide a complete reproducibility package including fine-tuned Qwen2.5-based models, experimental code, and detailed adaptation guidelines for Neo4j graph databases to facilitate adoption and encourage further research.

1.4 Related Work

The Text-to-Cypher task has advanced rapidly alongside developments in small language models and Graph RAG systems [1, 2]. The Text2Cypher-2024v1 dataset [2] serves as the primary benchmark, offering natural language–Cypher pairs across

diverse domains. Existing approaches predominantly rely on supervised fine-tuning of SLMs [2], synthetic data augmentation [3, 4], or reinforcement learning refinement [9]. Recent works have also explored multilingual aspects of Text-to-Cypher, evaluating foundational models on non-English questions while keeping the same Cypher output [10], and unified training paradigms that leverage structural transfer between Text-to-SQL and Text-to-Cypher via reinforcement learning [11]. For schema management in large graphs, Ozsoy et al. [5] employ heuristic keyword pruning and LLM-based filtering to reduce context length, though these methods often yield modest accuracy gains beyond full-schema prompting with capable SLMs. In the Text-to-SQL domain, draft-based schema linking [6, 6] and execution feedback mechanisms [7, 8] have shown substantial improvements, but their adaptation to graph-specific constraints—such as relationship directionality, variable-length paths, and pattern matching—has not been thoroughly explored. Our work addresses this gap through a training-free pipeline that adapts these proven techniques to Cypher generation. We extend draft-based schema linking to handle graph patterns by parsing MATCH clauses and relationship types via regex, yielding precise sub-schemas without requiring separate embedding-based retrieval or multiple LLM calls for schema selection. Unlike keyword-based filtering [5], our method leverages the model's own draft query as a signal for schema relevance. For execution-driven refinement, we adapt the self-correction loop from Text-to-SQL systems with two key innovations: (1) validation rules tailored to graph query constraints, and (2) automatic chain-of-thought hints that provide targeted diagnostic guidance for common Cypher error categories without requiring additional LLM calls. Our ablation studies confirm that schema linking reduces hallucinations while execution-driven self-correction substantially improves executability across both base and fine-tuned variants, demonstrating the value of systematic integration in a compute-efficient, training-free framework.

2 Materials and Methods

This section outlines our proposed two-phase methodology aimed at improving the Text-to-Cypher capabilities of open-source SLMs—specifically those with fewer than 14 billion parameters—on complex graph query tasks. The workflow begins with Phase I: Schema Linking via Draft Query Extraction, a process that generates an initial query sketch from the full schema and employs regex-based matching to extract a minimal, relevant sub-schema. Next, Phase II: Execution-Driven Self-Correction takes place at inference time, utilizing the filtered sub-schema in the prompt to produce Cypher queries and incorporating an iterative refinement loop based on syntax validation and runtime feedback from the graph engine. This structured approach systematically addresses challenges such as schema complexity, context window limitations, and query inaccuracies prevalent in smaller models.

2.1 Materials

2.1.1 Models

We evaluate our enhancement pipeline on four open-source SLMs representing different architectural designs and training paradigms. This selection enables comprehensive assessment across general-purpose instruction following, code-specialized reasoning, and task-specific fine-tuning approaches:

Base Models (Zero-shot)

- **Qwen2.5-7B-Instruct**—General-purpose instruction-tuned model from Alibaba Cloud's Qwen2.5 series, trained on diverse instruction-following tasks. Serves as our primary zero-shot baseline to evaluate the pipeline's effectiveness without domain-specific fine-tuning.
- **Qwen2.5-Coder-14B-Instruct**—Code-optimized variant with 14B parameters, featuring enhanced syntactic reasoning and structured output generation capabilities. Tests whether code pre-training provides inherent advantages for query generation tasks requiring precise syntax adherence.

Fine-tuned Models

- **neo4j/text2cypher-gemma-2-9b-it-finetuned-2024v1** (Gemma2-ft)—Gemma-2-9B fine-tuned on the Text2Cypher-2024v1 training set by Neo4j. This model serves as an established baseline from [2], representing the current state-of-the-art in supervised Text-to-Cypher approaches.
- **Qwen2.5-7B-Instruct-ft** (Our fine-tuned model)—We fine-tune Qwen2.5-7B-Instruct on the same Text2Cypher-2024v1 training set using LoRA (rank $r = 16$, scaling factor $\alpha = 32$) for 1 epoch with learning rate 2×10^{-4}. This model achieves 50.3% Execution Accuracy on the executable subset of the test set, demonstrating competitive performance with task-specific adaptation while maintaining efficient parameter usage.

All models, including both base and fine-tuned variants, are deployed with 4-bit quantization for memory-efficient inference on consumer-grade hardware. All experiments are conducted on Google Colab with Tesla T4 GPU (16GB VRAM), representing accessible computational resources for researchers and practitioners in resource-constrained environments. This diverse model selection enables systematic assessment across three key dimensions: (1) architectural variations (Qwen vs. Gemma families), (2) pre-training objectives (general instruction-following vs. code-specialized reasoning), and (3) training paradigms (zero-shot inference vs. supervised fine-tuning). The combination demonstrates our pipeline's broad applicability and complementary value to both base and task-adapted models.

2.1.2 Dataset

We evaluate our pipeline on the Neo4j Text2Cypher (2024v1) dataset [2], a comprehensive benchmark for the Text-to-Cypher task comprising 44,387 instances (39,554 for training and 4,833 for testing). This dataset aggregates and cleans examples from

multiple public sources, providing natural language questions paired with corresponding Cypher queries and graph schemas across diverse domains. Of the 4,833 test instances, 2,471 samples (51.1%) are associated with executable graph databases spanning 16 unique database schemas, enabling direct execution accuracy evaluation. The remaining 2,362 samples (48.9%) lack corresponding databases and are evaluated using textual metrics only (BLEU, ROUGE-L, Token F1).

Evaluation Limitations. Unlike Text-to-SQL benchmarks such as Spider [12], which provide over 200 diverse executable databases for comprehensive cross-domain evaluation, the Text2Cypher-2024v1 dataset includes only 16 fully executable graph instances. This constraint, inherent to current Text-to-Cypher benchmarking resources, restricts execution-based assessment to approximately half of the test set and limits comprehensive evaluation of model generalization across diverse schema structures. While this limitation affects statistical robustness of execution accuracy measurements, the available executable subset remains the standard benchmark for assessing functional correctness in Text-to-Cypher research. Despite these constraints, the dataset's complex queries—including multi-hop traversals, variable-length paths, and aggregations—make it a challenging and representative testbed for assessing model performance in realistic graph querying scenarios.

2.1.3 Evaluation Metrics

To comprehensively assess the performance of the proposed models, we employ **Execution Accuracy** as the primary metric to verify functional correctness [12], complemented by standard generation metrics (**BLEU** [13], **ROUGE-L** [14], and **Token F1**) to evaluate lexical similarity.

- **Execution Accuracy:** Since multiple valid Cypher queries can produce the same result for a given question (e.g., varying variable names or clause ordering), text-based matching is often insufficient. For this reason, we do not report Exact Match (EM), as it penalizes semantically equivalent queries that differ only in superficial syntactic choices, making it unsuitable for evaluating functional correctness in graph query generation. Therefore, we prioritize execution accuracy, which validates the query based on its actual output from the Neo4j database. We execute both the predicted query and the ground truth query on a single database instance, then compare their result sets after normalization (e.g., lowercasing strings, rounding numerics, removing duplicates). A prediction is considered correct in two scenarios:

  1. *Values Match:* The returned data from the predicted query is identical to the ground truth result set. The comparison is order-agnostic unless an `ORDER BY` clause is explicitly present in the ground truth query.
  2. *Subset Match:* To account for cases where the model retrieves the correct entities but projects fewer properties than the ground truth (e.g., returning only movie titles when the ground truth includes release years), we implement a column-mapping verification. A query is deemed correct if its result contains fewer columns than the ground truth, but each column can be matched to a corresponding ground truth column with identical values.

While this single-database evaluation approach may be susceptible to false positives when queries with different semantics coincidentally produce identical results [15], it remains the standard evaluation method for Text-to-Cypher benchmarks where generating multiple test database instances per schema is not feasible given current dataset constraints.

- **Text-based Metrics:** To measure the lexical overlap between generated and reference queries, we report:
  - **BLEU:** Measures n-gram precision (up to 4-grams) with smoothing, providing a proxy for syntactic similarity [13].
  - **ROUGE-L:** Computes the longest common subsequence (LCS) between prediction and reference, capturing structural similarity [14].
  - **Token F1:** Evaluates the precision and recall of individual tokens (keywords, schema elements) in the generated query, disregarding their order. This metric is particularly useful for measuring the correct retrieval of schema terms.

2.2 Method

Figure 1 provides an overview of the proposed workflow, illustrating the complete workflow from natural language input to executable Cypher output. The pipeline consists of two main phases: schema linking via draft query extraction, followed by execution-driven self-correction. These phases operate sequentially on SLMs to produce accurate Cypher queries without requiring additional training.

2.2.1 Schema Linking

Schema linking identifies the relevant subset of the database schema needed to answer a given natural language query. For large-scale knowledge graphs with hundreds of node labels and relationship types, directly feeding the entire schema to language models leads to context overflow and decreased generation quality. Our approach addresses this challenge by adopting a draft-based schema linking strategy inspired by recent advances in Text-to-SQL systems (e.g., [6, 16]).

Schema Representation with Examples

Similar to the approach in [7], we enhance schema representation by including sample values for each property. During system initialization, we precompute the complete schema structure along with representative examples: for each node label L and relationship type R, we extract all properties and sample one valid example value from the database. All sampled values are validated to ensure they are meaningful (non-null, appropriate length, not encoded binary data). This precomputation step ensures low latency during query processing, as the schema information is cached and readily available.

Draft-based Schema Linking

Instead of predicting relevant schema components directly from the natural language query, we adopt a two-stage approach inspired by [6], leveraging intermediate Cypher generation as a high-fidelity relevance signal.

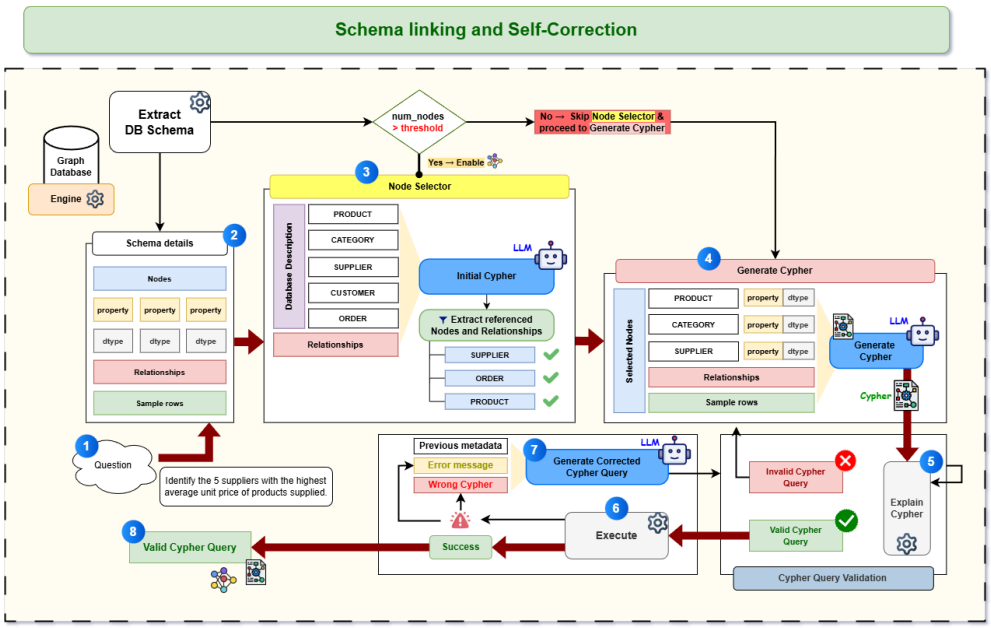

Fig. 1 Overview of the proposed workflow for Text-to-Cypher translation, illustrating the sequential integration of draft-based schema linking and execution-driven self-correction with automatic chain-of-thought hints.

In **Stage 1**, given a natural language query q and the complete schema $\mathcal{S}_{full}$ with property examples, we generate a draft Cypher query C_{draft} using the SLM:

$$C_{draft} = \text{SLM}(q, \mathcal{S}_{full}) \tag{1}$$

This initial generation uses the full schema to allow the model to reference any potentially relevant elements without prior filtering.

In **Stage 2**, we extract relevant node labels from C_{draft} using regex-based word boundary matching. For each node label L in the schema inventory, we apply a regular expression pattern that matches L as a complete word (case-insensitive) in the draft query:

$$\mathcal{N}_{draft} = \{L \in \mathcal{L}_{schema} \mid \text{regex_match}(L, C_{draft})\} \tag{2}$$

For schemas with few node labels (≤ 3), we skip filtering and retain the full schema to preserve complete context. Otherwise, we construct the refined sub-schema S_{linked} by including matched nodes and only the relationships that connect them:

$$S_{linked} = \{(L, P_L) \mid L \in \mathcal{N}_{draft}\} \cup \{R \mid \text{start}(R), \text{end}(R) \in \mathcal{N}_{draft}\} \tag{3}$$

Here, P_L denotes the properties associated with label L. A fallback mechanism reverts to $\mathcal{S}_{full}$ if $\mathcal{N}_{draft}$ is empty.

In **Stage 3**, the final Cypher query is generated using the focused schema:

$$C_{final} = \text{SLM}(q, S_{linked}) \tag{4}$$

This extraction strategy offers significant advantages: (1) *Self-referential relevance*—the model's draft naturally surfaces the schema elements it considers relevant, acting as an implicit reasoning step; (2) *Ambiguity resolution*—unlike direct keyword matching on the user question, the draft query resolves synonyms or indirect references into concrete schema labels; and (3) *Efficiency*—regex operations incur negligible computational overhead compared to embedding-based retrieval or syntactic parsing.

2.2.2 Execution-Driven Self-Correction

Execution-driven self-correction refines generated queries by leveraging direct feedback from the database engine, enabling the model to iteratively fix errors such as syntax violations, undefined labels, or mismatched patterns. In Text-to-Cypher tasks on large knowledge graphs, initial generations frequently contain errors that exact-match metrics cannot detect but execution reveals immediately. Our approach implements an iterative correction loop with automatic diagnostic hints.

Self-Correction Loop

We implement an iterative refinement process that attempts to execute the generated Cypher query and uses any resulting error message as corrective guidance. The procedure is formalized in Algorithm 1.

Algorithm 1 Cypher Self-Correction Loop (max_retries = 3)

```
C_current ← C_initial
retry ← 0
while retry < max_retries do
    (success, error) ← Execute(C_current, DB)
    if success then
        return C_current                          ▷ Query executed successfully
    end if
    hint ← GenerateCoTHint(error)                 ▷ Automatic hint generation
    P_corr ← BuildPrompt(q, S_linked, C_current, error, hint)
    C_current ← SLM(P_corr)
    retry ← retry + 1
end while
return C_current                                  ▷ Return last attempt
```

Automatic Chain-of-Thought Hints

To guide the model more effectively during correction, we incorporate an automatic hint mechanism inspired by chain-of-thought prompting techniques [17]. Before invoking the LLM for correction, the error message is analyzed using lightweight string

matching to identify the error category. Based on the matched category, a pre-written diagnostic hint with structured reasoning steps is appended to the correction prompt.

We define four error categories with corresponding hints:

- *Syntax errors* (matched by keywords: "syntax error", "invalid syntax"): Hints remind clause ordering rules—WHERE must precede RETURN, WITH cannot be the final clause, Cypher does not support GROUP BY—and provide step-by-step guidance to identify and relocate misplaced clauses.
- *Unknown labels or properties* (matched by: "unknown", "does not exist", "not found"): Hints direct the model to verify element names against the provided schema and distinguish between node properties and relationship properties.
- *Aliasing errors* (matched by: "must be aliased", "alias"): Hints explain WITH clause requirements and offer two approaches—aliasing each property explicitly or passing entire node variables.
- *Pattern/structure errors* (matched by: "pattern", "cannot conclude"): Hints clarify that queries must end with RETURN (or update clauses), and that SIZE() requires pattern comprehension syntax rather than simple path expressions.

Each hint follows a three-step structure: (1) analyze the error message to identify the specific issue, (2) review relevant Cypher rules and common mistakes, and (3) apply the fix systematically. This structured format guides the model through explicit reasoning rather than relying solely on implicit pattern recognition.

Novelty of Automatic CoT Hints

The automatic chain-of-thought hints mechanism represents a key technical contribution that differentiates our approach from existing self-correction techniques. Prior work on execution-driven self-correction [8] typically feeds raw error messages directly to the model, relying on the LLM's inherent reasoning capabilities to interpret and address errors. However, database error messages are often cryptic and lack actionable guidance, and smaller language models may lack the reasoning capacity to diagnose complex errors from minimal feedback.

Our automatic CoT hints address these limitations through a rule-based error analysis layer that operates between execution feedback and model prompting. This layer: (1) categorizes errors into Cypher-specific types using lightweight string matching, (2) generates structured reasoning steps that explicitly guide the model through diagnostic reasoning, and (3) incorporates domain-specific knowledge that general-purpose SLMs may lack. The mechanism achieves targeted guidance without requiring additional LLM inference calls, maintaining computational efficiency while improving correction success rates.

Our self-correction method offers several advantages: (1) *Precise error localization*—feedback comes directly from the database engine, pinpointing issues such as invalid syntax, unknown labels, or malformed patterns; (2) *Effective handling of common pitfalls*—the four error categories cover the most frequent Cypher mistakes observed in our experiments; (3) *Controlled iteration*—a fixed maximum number of retries (three) prevents infinite loops while allowing substantial accuracy gains;

(4) *End-to-end integration*—the process operates seamlessly after schema linking, producing a final executable Cypher query without intermediate outputs.

Our design prioritizes lightweight, Cypher-specific refinement by combining direct execution feedback with targeted automatic CoT hints that address common query construction errors—including clause ordering, schema element verification, aliasing requirements, and query structure validity—resulting in robust and efficient query generation.

3 Results and Discussion

This section evaluates the proposed workflow on the Text2Cypher-2024v1 dataset across four SLMs: two base models (Qwen2.5-7B-Instruct and Qwen2.5-Coder-14B-Instruct) and two fine-tuned specialists (Gemma2-ft and Qwen2.5-7B-Instruct-ft). We first present overall performance metrics, followed by detailed analysis of model-specific behaviors, component contributions, and error patterns.

3.1 Results

3.1.1 Overall Performance on Base SLMs

Table 1 compares the full workflow against zero-shot baselines on base SLMs. The workflow achieves consistent improvements across all metrics. For Qwen2.5-7B-Instruct, Execution Accuracy increases by 8.3 percentage points (20.84%→29.14%), while BLEU, ROUGE-L, and Token F1 improve by 3.19 points, 0.025, and 0.029, respectively. For Qwen2.5-Coder-14B-Instruct, gains are more modest: 3.84 points in Execution Accuracy (37.07%→40.91%) with smaller improvements in textual metrics. These results demonstrate that the workflow effectively enhances query generation quality in zero-shot settings without task-specific training.

Table 1 Performance comparison between the full workflow and zero-shot baseline on base SLMs. ↑ indicates improvement over baseline.

Method	Model	Exec. Acc.	BLEU	ROUGE-L	Token-F1
Baseline	Qwen2.5-instruct	20.84	49.80	0.643	0.735
	Qwen2.5-coder	37.07	60.13	0.718	0.817
Workflow	Qwen2.5-instruct	29.14↑	52.99↑	0.668↑	0.764↑
	Qwen2.5-coder	40.91↑	61.26↑	0.722↑	0.821↑

3.1.2 Error Pattern Analysis

Table 2 categorizes failure modes into three types: different results (queries that execute but return incorrect output), execution issues (runtime errors from malformed patterns or missing properties), and invalid syntax (unparseable queries). The workflow substantially reduces syntax errors: from 25.6% to 7.3% for Qwen2.5-7B-Instruct (18.3 point reduction) and from 13.6% to 3.2% for Qwen2.5-Coder-14B (10.4

point reduction). However, the proportion of different results increases (47.3%→55.3% for Qwen2.5-7B-Instruct), indicating that while the workflow successfully addresses syntactic validity, semantic correctness remains challenging.

Table 2 Error distribution on the Neo4j/Text2Cypher test set. Lower values are better for all categories.

Method	Model	Diff. Res.	Exec. Issues	Inv. Syntax
Baseline	Qwen2.5-instruct	47.3	6.1	25.6
	Qwen2.5-coder	42.9	6.3	13.6
Workflow	Qwen2.5-instruct	55.3	8.1	**7.3**
	Qwen2.5-coder	49.0	7.0	**3.2**

3.1.3 Component Ablation on Base SLMs

Tables 3 and 4 isolate the contribution of schema linking and self-correction. For Qwen2.5-7B-Instruct, the full workflow (29.14% EX) outperforms schema linking alone (27.68%), self-correction alone (27.03%), and the baseline (20.84%). Similar patterns emerge for Qwen2.5-Coder-14B: full workflow (40.91%) exceeds individual components (39.38% and 39.66%) and baseline (37.07%). Both components contribute synergistically, with schema linking reducing schema-level hallucinations and self-correction addressing syntactic errors.

Table 3 Ablation study on Qwen2.5-7B-Instruct showing synergistic effects of schema linking and self-correction.

Method	Exec. Acc.	BLEU	ROUGE-L	Token-F1
Full Workflow	**29.14**	**52.99**	**0.668**	**0.764**
Schema Linking	27.68	52.69	0.667	0.761
Self-Correction	27.03	51.83	0.661	0.757
Baseline	20.84	49.80	0.643	0.735

Table 4 Ablation study on Qwen2.5-Coder-14B confirming complementary benefits of both components.

Method	Exec. Acc.	BLEU	ROUGE-L	Token-F1
Full Workflow	**40.91**	**61.26**	**0.723**	**0.821**
Schema Linking	39.38	61.06	0.721	0.820
Self-Correction	39.66	60.46	0.720	0.819
Baseline	37.07	60.13	0.718	0.817

3.1.4 Performance on Fine-tuned SLMs

Table 5 presents results on fine-tuned specialists. Unlike base SLMs, these models show mixed behaviors. For Gemma2-ft, the baseline (37.11% EX) outperforms the full workflow (33.35%), while self-correction alone achieves 37.19%. For Qwen2.5-7B-Instruct-ft, both baseline and self-correction alone reach 50.3%, exceeding the full workflow (47.1%). This contrasts sharply with base SLM results, where the full workflow consistently achieves highest scores.

For Qwen2.5-7B-Instruct-ft, we report only baseline, self-correction, and full workflow results. Given that the full workflow (which includes schema linking) already shows performance degradation compared to the baseline (47.1% vs. 50.3%), we did not conduct separate schema linking evaluation, as the combined results sufficiently demonstrate the negative impact of schema filtering on this fine-tuned model.

Table 5 Ablation results on fine-tuned SLMs showing reduced effectiveness of schema linking.

Model	Method	Exec. Acc.	BLEU	ROUGE-L	Token-F1
Gemma2-ft	Baseline	37.11	70.74	0.791	0.863
	Self-Correction	37.19	71.35	0.795	0.866
	Schema Linking	32.98	68.46	0.778	0.853
	Full Workflow	**33.35**	**68.60**	**0.780**	**0.821**
Qwen2.5-ft	Baseline	50.3	70.00	0.778	0.861
	Self-Correction	50.3	69.68	0.771	0.851
	Full Workflow	**47.1**	**68.89**	**0.771**	**0.851**

3.2 Discussion

3.2.1 Why Base SLMs Benefit from Schema Linking

Base SLMs lack task-specific knowledge of graph schemas. When presented with complete schemas containing dozens of node labels and relationship types, these models struggle to identify relevant elements, frequently hallucinating non-existent schema components or selecting inappropriate relationships. Schema linking addresses this by generating a draft query that implicitly reveals which schema elements the model considers relevant, then extracting mentioned node labels via lightweight regex matching (word boundary detection) to construct a focused sub-schema.

This approach offers two advantages over full-schema prompting. First, it reduces context length, enabling models with limited context windows (typically 4k–8k tokens) to process the question and schema without truncation. Second, it eliminates irrelevant schema elements that might confuse the model or dilute attention. The 8.3 point improvement for Qwen2.5-7B-Instruct demonstrates that even relatively small models can achieve substantial gains when provided with appropriately linked context.

The more modest improvement for Qwen2.5-Coder-14B (3.84 points) suggests that code-specialized pre-training provides inherent advantages in structural reasoning. Code models are trained to understand hierarchical structures, type systems, and compositional patterns—capabilities that transfer to graph schema comprehension. Consequently, these models derive less marginal benefit from explicit schema linking than general-purpose instruction models.

3.2.2 Why Fine-tuned SLMs Show Reduced Gains from Schema Linking

The counterintuitive results on fine-tuned models—where schema linking reduces performance relative to baselines—stem from a critical limitation of the evaluation dataset. Only 16 unique graph databases provide executable instances for Execution Accuracy measurement, and these schemas overlap substantially with the Text2Cypher-2024v1 training set used for fine-tuning. This overlap creates a scenario where fine-tuned models have memorized schema-specific patterns during training.

When fine-tuned models are presented with full schemas at inference time, they effectively leverage this learned knowledge. The models have internalized common label combinations, frequent relationship patterns, and typical property usage for each schema. However, schema linking dynamically extracts a sub-schema based on the draft query, which may exclude elements the model expects. This mismatch between training context (full schemas) and filtered inference context (sub-schemas) disrupts learned associations.

For example, if a model was trained to recognize that `Person` nodes frequently connect to `Movie` nodes via `ACTED_IN` relationships in a particular schema, but the schema linking step filters out `Movie` because it was not mentioned in the draft, the model may fail to generate the correct query despite having the necessary knowledge. This explains why Gemma2-ft with schema linking (32.98% EX) underperforms the baseline (37.11%): the filtering process removes contextual cues that the model relies on.

This finding reveals an important trade-off: schema linking is most valuable when models lack prior schema knowledge (base SLMs in zero-shot settings) but can be detrimental when models have been trained to exploit full schema context (fine-tuned specialists on familiar schemas). The limited diversity of executable test databases (16 schemas) amplifies this effect, as models can overfit to specific patterns that would not generalize to broader schema distributions.

3.2.3 Self-Correction Remains Effective Across All Models

Despite the mixed results for schema linking on fine-tuned models, self-correction demonstrates consistent benefits. Table 2 shows that syntax errors decrease dramatically for base SLMs (25.6%→7.3% for Qwen2.5-7B-Instruct), and Table 5 reveals that self-correction alone often matches or exceeds baseline performance for fine-tuned models (37.19% vs. 37.11% for Gemma2-ft).

Self-correction addresses syntactic and structural errors that arise during query generation, regardless of the model's prior training. Even fine-tuned models, which

have learned correct query patterns during supervised training, occasionally generate malformed queries due to ambiguous natural language inputs, complex traversal requirements, or edge cases not covered in training data. The execution-driven feedback loop identifies these errors through direct database interaction, while automatic chain-of-thought hints provide targeted diagnostic guidance for common Cypher error patterns.

The automatic hints mechanism—which analyzes error messages and appends structured reasoning steps—proves particularly valuable. Rather than requiring the model to interpret generic database error messages, the system provides explicit guidance such as "Check clause ordering: WHERE must precede RETURN" or "Verify aliasing: WITH clause requires explicit variable names." This targeted assistance enables models to localize and correct errors more efficiently than with raw error feedback alone.

3.2.4 Synergistic Effects and Component Interactions

The ablation studies on base SLMs (Tables 3 and 4) reveal that schema linking and self-correction address complementary aspects of query generation. Schema linking reduces schema-level hallucinations—preventing the model from generating queries with non-existent node labels or relationship types. Self-correction addresses syntactic and semantic errors that emerge during query construction, including clause ordering violations, missing aliases, and malformed pattern structures.

The full workflow achieves higher scores than either component alone because the two mechanisms operate at different stages and target distinct error types. Schema linking establishes a clean foundation by filtering irrelevant schema elements, enabling the model to focus attention on relevant graph structures. Self-correction then refines the resulting query through iterative validation and execution feedback, ensuring both syntactic correctness and semantic alignment with the database schema.

For Qwen2.5-7B-Instruct, the gain from combining both components (29.14% EX) is lower than the sum of individual improvements (27.68% + 27.03% − 20.84% baseline = 33.87% hypothetical), indicating subadditive interaction rather than independent effects. This subadditive behavior likely reflects the fact that schema linking and self-correction address overlapping error types: schema linking reduces certain errors that self-correction would otherwise need to address, resulting in diminished marginal gains when both are applied together.

3.2.5 Technical Contributions and Relation to Prior Work

It is important to clarify the novel aspects of our work relative to prior Text-to-SQL and Text-to-Cypher research. While schema linking and self-correction are established techniques, our contributions lie in their systematic integration and the introduction of automatic diagnostic hints.

Draft-Guided Schema Linking

Our schema linking approach leverages the model's own draft query as a relevance signal, distinguishing it from SQL-focused methods that typically use embedding-based retrieval or rule-based extraction from natural language questions [6, 16]. By generating an initial Cypher query and extracting mentioned node labels via keyword matching, we obtain an implicit relevance signal: the labels the model chooses to reference in its draft likely correspond to schema components needed for the final query. This self-referential approach avoids the need for separate retrieval modules and naturally handles cases where natural language questions use synonyms or indirect references to schema elements.

The keyword-based extraction, while simpler than full Cypher parsing, proves effective because draft queries typically mention relevant node labels explicitly. Our experiments show that this approach achieves an 8.3 percentage point improvement in Execution Accuracy for base models, demonstrating that simple extraction combined with draft-guided relevance provides substantial benefits.

Automatic CoT Hints as a Novel Mechanism

The automatic chain-of-thought hints mechanism is a technical innovation not present in prior self-correction work. While DIN-SQL [8] and similar approaches rely on raw error feedback, our method introduces an intermediate analysis layer that generates structured reasoning guidance. The four error categories (syntax, unknown elements, aliasing, pattern structure) cover the most frequent Cypher pitfalls observed in our experiments, and the step-by-step hint format provides explicit diagnostic reasoning that smaller models may not derive from cryptic error messages alone.

This mechanism is also extensible: new error patterns can be added by defining additional matching rules and hint templates, enabling continuous improvement as new Cypher pitfalls are identified. The 70% reduction in syntax errors (from 25.6% to 7.3% for Qwen2.5-7B-Instruct) demonstrates the effectiveness of targeted diagnostic guidance.

Systematic Evaluation Across Training Paradigms

Prior Text-to-Cypher work has primarily evaluated either base models in zero-shot settings or fine-tuned models in isolation [2, 4]. Our study systematically compares enhancement techniques across both paradigms, revealing the critical trade-off between schema memorization and dynamic schema linking. This finding—that schema linking benefits base models but can harm fine-tuned specialists on familiar schemas—has practical implications: practitioners should apply schema linking selectively based on model training history and target schema familiarity.

3.2.6 Error Distribution Shifts and Semantic Challenges

Table 2 reveals an important pattern: while the workflow dramatically reduces invalid syntax errors, the proportion of different results increases (47.3%→55.3% for Qwen2.5-7B-Instruct). This shift indicates that the workflow successfully converts unparseable queries into executable queries, but many of these queries still produce incorrect outputs due to logical errors.

Different results stem from semantic issues such as incorrect relationship directionality, inappropriate filtering conditions, or misaligned aggregation logic. These errors cannot be detected through syntax validation or execution feedback alone—they require comparing query outputs against expected results. While our automatic chain-of-thought hints address common syntactic patterns (clause ordering, aliasing requirements), they do not currently handle semantic validation.

This limitation points to future work: semantic validation mechanisms that compare query results against ground truth or example-based verification could further improve accuracy. Techniques such as result-aware prompting, where the model is shown expected output formats, or execution trace analysis, where intermediate query steps are validated, may help address this remaining challenge.

3.2.7 Dataset Limitations and Generalization

The primary limitation of this evaluation is the restricted number of executable graph databases (16 unique schemas, covering 51.1% of the test set). This constraint affects both statistical robustness and generalizability of findings. Execution Accuracy measurements are computed over approximately 2,471 samples, which, while substantial, represent only half the test set and are concentrated in a limited number of schema structures.

The limited schema diversity particularly impacts fine-tuned model evaluation. With only 16 schemas, fine-tuned models can memorize schema-specific patterns that would not transfer to broader distributions. The performance degradation observed with schema linking on fine-tuned models may partially reflect this overfitting: models trained on full schemas for these specific 16 databases perform worse when presented with dynamically filtered sub-schemas, but this behavior might not generalize to unseen schemas where full-schema memorization is impossible.

Compared to Text-to-SQL benchmarks like Spider, which provides over 200 diverse databases, the Text2Cypher-2024v1 dataset's limited executable instances restrict comprehensive cross-domain evaluation. Future work should prioritize expanding graph database benchmarks with more diverse schemas, ideally including databases with varying sizes, domains, and structural complexities. Such expansion would enable more robust assessment of schema linking effectiveness and reduce the risk of schema-specific overfitting.

4 Conclusion

This work introduces a training-free workflow that combines schema linking and execution-driven self-correction to improve Text-to-Cypher query generation for SLMs in the 7B–14B parameter range. Experimental results show that the proposed approach significantly benefits base SLMs in zero-shot settings, achieving gains of up to 8.3 percentage points in Execution Accuracy (e.g., from 20.84% to 29.14% for Qwen2.5-7B-Instruct). The improvement mainly comes from schema linking, which reduces irrelevant schema elements and context length, and from self-correction, which effectively fixes syntactic and structural errors using automatic chain-of-thought hints derived from execution feedback.

Surprisingly, schema linking provides limited or even negative effects on fine-tuned models. We attribute this to considerable schema overlap between the training and test sets (only 16 executable databases), allowing fine-tuned models to memorize patterns associated with full schemas. When schema linking filters out certain elements at inference time, it disrupts these learned patterns, leading to lower performance compared to using the full schema.

Despite the limited schema diversity in the evaluation, self-correction consistently improves performance across both base and fine-tuned models with low computational overhead. This makes it a practical component for real-world deployment. The training-free nature of the full workflow is particularly useful in scenarios where task-specific fine-tuning data or resources are unavailable, such as querying private enterprise knowledge graphs or domain-specific graph databases.

Several directions can be explored in future work: (1) developing mechanisms for semantic validation to reduce logical errors that current syntax-based correction cannot address, (2) designing adaptive schema linking strategies that better suit fine-tuned models, (3) expanding benchmarks with more diverse executable graph databases, and (4) integrating the workflow into Graph RAG systems to support reliable querying in production environments. These improvements would further enhance the applicability of SLMs for structured graph querying tasks.

Acknowledgments. This research was supported by The Vietnam National University Ho Chi Minh City - University of Information Technology's Scientific Research Support Fund.

References

[1] Peng, B., et al.: Graph retrieval-augmented generation: A survey. arXiv preprint arXiv:2408.08921 (2024)

[2] Ozsoy, M.G., Messallem, L., Besga, J., Minneci, G.: Text2cypher: Bridging natural language and graph databases. arXiv preprint arXiv:2412.10064 (2024)

[3] Zhong, Z., et al.: Synthet2c: Generating synthetic data for fine-tuning large language models on the text2cypher task. arXiv preprint arXiv:2406.10710 (2024)

[4] Tiwari, A., Malay, S.K.R., Yadav, V., Hashemi, M., Madhusudhan, S.T.: Autocypher: Improving llms on cypher generation via llm-supervised generation-verification framework. In: Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers) (2025)

[5] Ozsoy, M.G.: Enhancing text2cypher with schema filtering. arXiv preprint arXiv:2505.05118 (2025)

[6] Lei, Y., Li, Y., Yang, X., Wang, Y., Zhang, M., Yang, K., Zhang, L., Liu, C.: Sql-to-schema enhances schema linking in text-to-sql. arXiv preprint arXiv:2405.09593 (2024)

[7] Le Gia, K., Le Quoc, K., Nguyen Minh, N., Nguyen Dinh, T.: Enhancing text-to-sql capabilities of small language models via schema context enrichment and self-correction. CTU Journal of Innovation and Sustainable Development **17**(Special issue: ISDS) (2025) https://doi.org/10.22144/ctujoisd.2025.058

[8] Pourreza, M., Rafiei, D.: Din-sql: Decomposed in-context learning of text-to-sql with self-correction. arXiv preprint arXiv:2304.11015 (2023)

[9] Tran, M., et al.: Refining text2cypher on small language model with reinforcement learning leveraging semantic information. Some journal or arXiv (2025)

[10] Ozsoy, M.G., Tai, W.: Text2Cypher Across Languages: Evaluating Foundational Models Beyond English. arXiv preprint arXiv:2506.21445 (2025)

[11] Zhang, X., et al.: Struct-llm: Structured transfer for text-to-sql and text-to-cypher. arXiv preprint (2025). Preprint (details unavailable)

[12] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., *et al.*: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In: Proceedings of the 2018 Conference on Empirical Methods on Natural Language Processing, pp. 3911–3921 (2018)

[13] Papineni, K., Roukos, S., Ward, T., Zhu, W.-J.: Bleu: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, pp. 311–318. Association for Computational Linguistics, Philadelphia, Pennsylvania (2002)

[14] Lin, C.-Y.: Rouge: A package for automatic evaluation of summaries. In: Text Summarization Branches Out: Proceedings of the ACL-04 Workshop, pp. 74–81. Association for Computational Linguistics, Barcelona, Spain (2004)

[15] Zhong, R., Yu, T., Klein, D.: Semantic evaluation for text-to-sql with distilled test suites. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 396–411 (2020)

[16] Li, Z., Wang, X., Zhao, J., Yang, S., Du, G., Hu, X., Zhang, B., Ye, Y., Li, Z., Zhao, R.: Rsl-sql: Robust schema linking in text-to-sql generation. arXiv preprint arXiv:2411.00073 (2024)

[17] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems **35**, 24824–24837 (2022)